

RHYNK

"Every Chat Has a Rhythm."

High-Level Design (HLD)

- Type:** System Architecture Overview — for anyone getting oriented on the project
- Scope:** Services, data flow, tech stack, non-functional goals, build roadmap
- Companion:** See LLD document for schemas, API contracts, algorithms, sequence diagrams
- Date:** July 2026

Table of Contents

- [1](#) What Rhynek is — product summary
- [2](#) High-level system architecture
- [3](#) Services & responsibilities
- [4](#) Data layer at a glance
- [5](#) External dependencies
- [6](#) Non-functional goals
- [7](#) Build roadmap (phases)

1. What Rhynek is — product summary

Rhynek is a web-first, WhatsApp/Telegram-style messaging platform with one core differentiator: **synchronized group music listening inside a chat** — a "music room" where everyone in a conversation hears the same song at the same position at the same time, with one member acting as DJ. Primary market is India; the two product pillars (chat + music) are equally important, not chat-with-a-music-bolt-on. This document (HLD) is the map — what exists and how the pieces talk to each other. For the actual table schemas, API request/response contracts, and the algorithms that make music-sync and auth work correctly under real-world network conditions, see the companion **LLD document**.

2. High-level system architecture

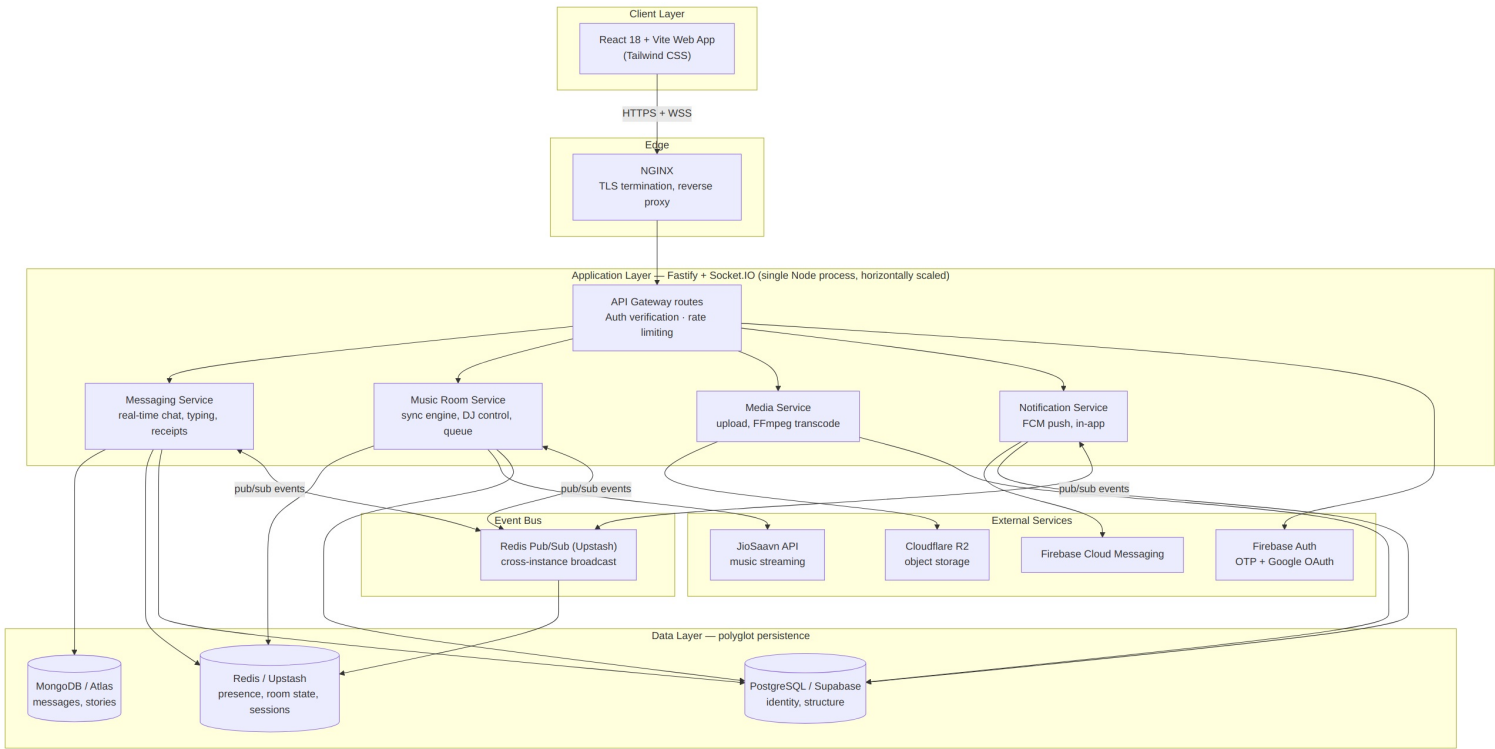


Figure 1 — full system architecture: client, edge, application services, event bus, data layer, external dependencies

Everything in the **Application Layer** runs as Fastify route/plugin modules inside a single Node.js process type (not separate microservices at this stage) — "Messaging Service," "Music Room Service," etc. are logical divisions of responsibility within one deployable unit, horizontally scaled behind NGINX. This is a deliberate MVP simplification: splitting into real microservices before there's a scaling reason to would add deployment complexity with no benefit yet.

3. Services & responsibilities

Service	Technology	Responsibility
Client (Web)	React 18 + Vite + Tailwind	UI, local state, Socket.IO client, audio playback
API Gateway	Fastify + NGINX	Auth verification, rate limiting, request routing
Messaging Service	Fastify + Socket.IO	Real-time chat, E2E relay, typing, read receipts
Music Room Service	Fastify + Socket.IO	Sync engine, DJ control, queue, NTP-style offset calc
Media Service	Fastify + FFmpeg + Bull	File upload, audio transcode, R2 storage
Notification Service	Fastify + FCM	Web push, mobile push, in-app notifications
Event Bus	Redis Pub/Sub (Upstash)	Cross-instance events, presence, room state broadcast

Each service above maps directly to a folder under `server/src/modules/` in the actual codebase — see the LLD document's folder-structure section for the exact mapping.

4. Data layer at a glance

Store	Holds	Why
PostgreSQL (Supabase)	Users, conversations, membership, music room metadata, songs, playlists, notifications, calls	Relational, high-integrity, needs real joins and transactions
MongoDB (Atlas)	Messages, stories	Highest write volume, variably-shaped documents, append-heavy
Redis (Upstash)	Presence, live room state, typing, rate limits, OTP, sessions	Sub-second mutation, TTL-based expiry, ephemeral by nature

Full schemas, ER diagrams, and the reasoning behind every table live in the **Database Design deep-dive PDF** already produced for this project — this HLD only summarizes the split at a glance.

5. External dependencies

Service	Used for	Free-tier note
Firebase Auth	Phone OTP, Google OAuth	10k verifications/mo free
Firebase Cloud Messaging (FCM)	Push notifications, web + mobile	Free, no meaningful cap
JioSaavn (unofficial API)	Music search, streaming, charts, radio	Proxied server-side — client never calls it directly
Cloudflare R2	User-uploaded audio, images, story media	Free egress, unlike S3 — a deliberate cost decision

6. Non-functional goals

- **Sync accuracy:** music room playback offset under 500ms across members, self-correcting every 30s
- **Security:** JWT access + rotating refresh tokens, E2E-relayed text messages, per-IP and per-user rate limiting on every sensitive route
- **Compliance:** all infra pinned to `ap-south-1` (Mumbai) for data residency; soft-delete + anonymization for right-to-be-forgotten
- **Cost:** every managed service chosen has a genuinely usable free tier at MVP scale — the architecture is explicitly designed to run at near-zero infra cost until real usage justifies upgrading a specific tier
- **Scalability path:** each data store has an identified, low-disruption next step (Postgres read replicas, Mongo sharding on `conversation_id`, Redis tier upgrade) rather than a rewrite

7. Build roadmap (phases)

Phase	Focus	Why this order
0	Foundation — env setup, DB connections, plugin wiring	A broken connection or wrong plugin order causes silent failures elsewhere; fix once, never debug mid-feature
1	Identity & Security — auth, users, devices	Every other table's foreign keys depend on <code>users</code> existing first
2	Core Communication — conversations, membership	A message has no home without a conversation; <code>conversation_id</code> is the contract every message document depends on
3	Messaging — real-time chat over Socket.IO	Flagged as highest-risk phase (WebSocket scaling, ordering, offline delivery)
4	Music Rooms — sync engine, DJ mode, queue	Reuses the Socket.IO gateway built for messaging rather than standing up a second one
5	Media, Notifications, Calls, Hardening	Notifications are an observer of every module above — they need message/room/call events to already exist

Estimated effort: 11–13 weeks across all five phases, per the current PRD estimate.